

UNIVERSIDAD AUTÓNOMA DE CHIAPAS

Facultad de Ciencias en Física y Matemáticas · IDTS

MedIA — Expediente Clínico Electrónico

Distrito de Salud I · Tuxtla Gutiérrez, Chiapas

DOCUMENTO 2

Arquitectura General del Sistema

8° Semestre · IDTS · UNACH · Cómputo Distribuido · Enero–Junio 2026

1. Stack Tecnológico y Justificación

Cada tecnología del stack fue seleccionada con criterios técnicos específicos para el contexto de un sistema de salud con 40 entidades, normativa mexicana estricta y despliegue en zona geográfica con conectividad variable.

1.1 Frontend — React + Vite + shadcn/ui + DM Sans

Tecnología	Justificación técnica
React 18	Virtual DOM eficiente para interfaces con alta densidad de datos (tablas de expediente, listados CIE-10, bitácora de auditoría). Ecosistema maduro con shadcn/ui y React Router.
Vite	HMR (Hot Module Replacement) extremadamente rápido vs Create React App. Build optimizado con tree-shaking. Configuración nativa para variables de entorno VITE_* separadas de secrets del servidor.
shadcn/ui	Componentes accesibles (ARIA) y estilizables sin dependencia de runtime CSS-in-JS. Los componentes son código propio (no node_modules), lo que permite ajustes normativos como el modal de 2FA y el stepper SOAP.
DM Sans	Fuente diseñada para densidad de información. Excelente legibilidad en pantallas de baja resolución de consultorios. Variante óptica para tamaños pequeños. Reduce fatiga visual en turnos de 8+ horas (alineado con NOM-035-STPS-2018).
JWT en memoria	El access token se almacena ÚNICAMENTE en useState del AuthContext, nunca en localStorage ni sessionStorage. Mitiga ataques XSS que extraen tokens del storage. Complementado con middleware de sanitización en el backend.

Tabla 1.1 — Justificación del stack frontend

1.2 Backend — Python + FastAPI + SQLAlchemy + Pydantic

Tecnología	Justificación técnica
Python 3.11	Rendimiento mejorado (+10–60% vs 3.10) con better error messages. Ecosistema médico/científico consolidado. pyotp para 2FA, WeasyPrint para PDFs, bleach para sanitización XSS.
FastAPI	Documentación automática en Swagger/OpenAPI: esencial para que el equipo comparta contratos de API. Validación de tipos con Pydantic en cada request/response. async nativo para operaciones concurrentes (generación PDF + consulta BD simultáneas).
SQLAlchemy 2.x	ORM con soporte async para PostgreSQL. Queries tipadas que el middleware de auditoría puede interceptar. Permite mezclar ORM y SQL nativo para las vistas SQL (v_paciente_basico, v_signos_encuentro).
Pydantic v2	Validación de esquemas de request/response en cada endpoint. Permite definir restricciones normativas como campos obligatorios (cédula profesional, motivo de consulta) directamente en el schema.
Argon2id	Algoritmo de hashing de contraseñas ganador del Password Hashing Competition (2015). Más seguro que bcrypt contra ataques GPU. Configurable en tiempo, memoria y paralelismo según el hardware del servidor.

WeasyPrint	Generación de PDFs desde HTML/CSS en Python. Los PDFs del expediente (nota SOAP, receta, solicitud, referencia) se generan desde las plantillas HTML del sistema, garantizando autoría verificable y timestamp del servidor (NOM-151).
------------	--

Tabla 1.2 — Justificación del stack backend

1.3 Base de Datos — PostgreSQL 15 en Azure

Tecnología	Justificación técnica
PostgreSQL 15	Tipos nativos UUID, TIMESTAMPTZ, ENUM, JSONB, INET utilizados en el modelo de 40 entidades. Triggers PL/pgSQL para inmutabilidad de notas (tr_notes_protection) y bitácora (tr_audit_no_changes). Soporte para UUID v4 nativo con gen_random_uuid().
Azure Database for PostgreSQL	Servicio gestionado: backups automáticos con PITR (Point-In-Time Recovery) de 5 min de granularidad. Replicación síncrona y failover automático sin código propio. Escalado vertical sin migración de datos. Cumple SOC 2 Type II.
Azure Blob Storage	Almacena ÚNICAMENTE los PDFs de resultados de laboratorio (origen externo) y documentos de tutores. Los documentos del sistema se generan on-demand y no se persisten. SAS tokens de 15 min para acceso temporal seguro.

Tabla 1.3 — Justificación de la plataforma de base de datos

1.4 Infraestructura Azure

Servicio	Rol en la arquitectura
Azure App Service	Hospeda el backend FastAPI (Python 3.11). Escalado automático horizontal. Variables de entorno gestionadas en Azure Portal (sin secrets en código).
Azure Static Web Apps	Hospeda el frontend React + Vite. CDN integrado con enrutamiento SPA. Deploys automáticos desde rama main de GitHub.
Azure Front Door + CDN	Enrutamiento global con menor latencia para unidades rurales del Distrito. Cache de catálogos de solo lectura (cat_cie10, cat_medicamentos) en nodos regionales. WAF (Web Application Firewall) contra XSS y SQLi.
Azure Monitor	Telemetría de errores y latencia del backend. Alertas para eventos de seguridad (intentos de login fallidos masivos, accesos fuera de horario). Logs complementarios a la auditoria_accesos de PostgreSQL.
Docker (solo dev)	En desarrollo local: ÚNICAMENTE para el contenedor de PostgreSQL. El backend y frontend se ejecutan directamente en el host. En producción no hay contenedores propios; Azure gestiona el runtime.

Tabla 1.4 — Servicios de infraestructura Azure y sus roles

2. Entorno de Desarrollo con Docker

El entorno de desarrollo local usa un único contenedor Docker para PostgreSQL. El backend (FastAPI) y el frontend (Vite) corren directamente en el sistema operativo del desarrollador para máxima velocidad de iteración.

docker-compose.yml — PostgreSQL de Desarrollo

```
# docker-compose.yml — MedIA Development
# SOLO para desarrollo local. En producción usar Azure Database for PostgreSQL.
version: '3.9'
services:
  postgres:
    image: postgres:15-alpine
    container_name: media_db_dev
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${POSTGRES_DB:-media_db}
      POSTGRES_USER: ${POSTGRES_USER:-media_dev}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "5432:5432"
    volumes:
      - media_postgres_data:/var/lib/postgresql/data
      - ./database/01_schema.sql:/docker-entrypoint-initdb.d/01_schema.sql
      - ./database/02_triggers.sql:/docker-entrypoint-initdb.d/02_triggers.sql
      - ./database/03_seeds.sql:/docker-entrypoint-initdb.d/03_seeds.sql
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U ${POSTGRES_USER:-media_dev}']
      interval: 10s
      timeout: 5s
      retries: 5
volumes:
  media_postgres_data: {}
```

Importante: Los scripts SQL en /docker-entrypoint-initdb.d/ se ejecutan automáticamente solo cuando el volumen está vacío (primera vez). Para re-inicializar: `docker-compose down -v && docker-compose up`.

3. Estructura Completa de Carpetas

3.1 Backend — Python / FastAPI

```

backend/
├── app/
│   ├── main.py                # Registro de routers, CORS, middleware stack
│   ├── config.py              # Settings Pydantic (carga .env, valida vars)
│   ├── database.py            # AsyncEngine + AsyncSession factory
│   ├── middleware/
│   │   ├── auth.py            # Valida JWT, verifica blacklist en sesiones_invalidas
│   │   ├── audit.py           # Registra CADA request en auditoria_accesos (Req 4 Forense)
│   │   └── sanitize.py        # Bleach (XSS) + regex (SQLi) en parámetros de entrada
│   ├── models/                # SQLAlchemy ORM — una clase por entidad
│   │   ├── base.py            # Base declarativa + metadata
│   │   ├── personas.py        # PersonaModel, CatEstadoModel, etc.
│   │   ├── usuarios.py        # UsuarioSistemaModel, RolModel, PermisoRolModel
│   │   ├── pacientes.py       # PacienteModel, AlergiaModel, AntecedentesModel
│   │   ├── clinico.py         # EncuentroModel, NotaMedicaModel, NotaSoapDetalleModel
│   │   ├── seguridad.py       # AuditoriaAccesosModel, IncidenteModel
│   │   └── ...                # (un archivo por grupo de entidades relacionadas)
│   ├── schemas/               # Pydantic v2 — schemas de request y response
│   │   ├── auth.py            # LoginIn, TokenOut, TotpSetupOut, TotpVerifyIn
│   │   ├── pacientes.py       # PacienteCreateIn, PacienteOut, ExpedienteOut
│   │   ├── clinico.py         # EncuentroCreateIn, NotaSoapIn, FirmaOut
│   │   └── ...                # (schema separado por dominio)
│   ├── routers/               # FastAPI APIRouter — rutas por módulo
│   │   ├── auth.py            # /auth/login, /auth/refresh, /auth/logout, /auth/2fa/*
│   │   ├── personas.py        # /personas
│   │   ├── pacientes.py       # /pacientes/{id}/expediente
│   │   ├── encuentros.py      # /encuentros/{id}/cerrar
│   │   ├── notas.py           # /notas, /notas/{id}/firmar, /notas/{id}/pdf
│   │   ├── signos.py          # /signos-vitales
│   │   ├── diagnosticos.py     # /diagnosticos, /catalogos/cie10
│   │   ├── prescripciones.py   # /prescripciones
│   │   ├── laboratorio.py     # /solicitudes-estudio, /resultados-laboratorio
│   │   ├── referencias.py     # /referencias
│   │   ├── auditoria.py       # /auditoria, /incidentes
│   │   ├── admin.py           # /usuarios, /establecimientos, /roles
│   │   └── catalogos.py       # /catalogos/medicamentos, /catalogos/cie10
│   ├── services/              # Lógica de negocio — sin acceso directo a BD
│   │   ├── auth_service.py     # JWT, 2FA, bloqueo de cuentas
│   │   ├── paciente_service.py # CRUD + generación numero_expediente
│   │   ├── expediente_service.py # 4 Reglas de Negocio
│   │   ├── clinico_service.py  # Firma de notas, validación inmutabilidad
│   │   ├── referencias_service.py # SRC: emitir, aceptar, contrarreferir
│   │   └── acceso_service.py   # Validación RBAC cross-establecimiento
│   ├── utils/
│   │   ├── security.py         # Argon2id hash/verify, JWT encode/decode
│   │   ├── pdf_generator.py    # WeasyPrint: nota SOAP, receta, solicitud, referencia
│   │   └── file_handler.py     # Azure Blob Storage: upload + SHA-256 + SAS token
│   └── database/
│       ├── 01_schema.sql       # CREATE TABLE de las 40 entidades + ENUMs + índices
│       ├── 02_triggers.sql     # 6 triggers: inmutabilidad, historial, bloqueo, incidentes
│       └── 03_seeds.sql        # Catálogos INEGI, CIE-10, admin@media.local

```

```

├── tests/
│   ├── conftest.py          # Fixtures: AsyncSession de prueba, datos mock
│   ├── test_auth.py         # Tests de login, 2FA, bloqueo por intentos
│   └── test_reglas_negocio.py # Tests de las 4 Reglas de Negocio
├── .env.example             # Plantilla de variables de entorno (sin secrets)
├── requirements.txt          # Dependencias Python con versiones fijadas
└── Dockerfile               # (solo para referencia – en prod usa Azure App Service)

```

3.2 Frontend — React / Vite

```

frontend/
├── src/
│   ├── api/                # Funciones de llamada a la API REST
│   │   ├── auth.js         # login(), refresh(), logout(), setup2FA()
│   │   ├── pacientes.js    # getPacientes(), getPaciente(id), createPaciente()
│   │   ├── clinico.js      # getEncuentro(), crearNota(), firmarNota(id)
│   │   └── ...              # (un archivo por dominio)
│   ├── components/
│   │   ├── layout/
│   │   │   ├── Sidebar.jsx # Grupos 'Clínica / Sistema', rol del usuario visible
│   │   │   └── TopBar.jsx  # Número de expediente, CLUES, turno (req. NOM-004)
│   │   ├── forms/          # Formularios reutilizables por módulo
│   │   └── tables/         # Tablas paginadas con filtros
│   ├── pages/
│   │   ├── auth/           # LoginPage.jsx (credenciales + OTP 2FA)
│   │   ├── dashboard/      # DashboardPage.jsx (condicional por rol)
│   │   ├── pacientes/      # PacientesPage, PacienteFichaPage
│   │   ├── consulta/       # ConsultaPage.jsx (stepper 5 pasos SOAP)
│   │   ├── expediente/     # ExpedientePage.jsx (historial + notas + antecedentes)
│   │   ├── referencias/    # ReferenciasPage.jsx
│   │   ├── documentos/    # DocumentosPage.jsx (descarga de PDFs)
│   │   ├── auditoria/     # AuditoriaPage.jsx (bitácora + incidentes)
│   │   ├── admin/         # UsuariosPage, EstablecimientosPage, RolesPage
│   │   └── seguridad/     # SeguridadPage.jsx (sesiones inválidas, estadísticas)
│   ├── hooks/
│   │   ├── useAuth.js      # Lee AuthContext: user, token, login, logout
│   │   └── usePaciente.js  # Cache y fetching del paciente activo
│   ├── context/
│   │   └── AuthContext.jsx # JWT en memoria (useState), refresh silencioso
│   └── utils/
│       └── permissions.js  # checkPermission(rol, modulo, accion): true/false
├── .env.example            # VITE API_URL, VITE_APP_NAME
├── vite.config.js          # Proxy /api → backend (dev), Build settings
└── index.html              # Entry point – carga el bundle Vite

```

Figura 3.1 y 3.2 — Estructura de carpetas del proyecto MedIA

4. Variables de Entorno (.env.example completo)

El archivo .env.example documenta TODAS las variables de entorno requeridas sin exponer valores reales. Cada desarrollador crea su .env local copiando este archivo. Los valores de producción se configuran en Azure App Service como Application Settings.

Backend — .env.example

```
# =====
# MedIA — Backend Environment Variables
# Copiar a .env y completar con valores reales.
# NUNCA commitear el .env al repositorio.
# =====

# — ENTORNO —
APP_ENV=development           # development | staging | production
APP_NAME=MedIA-ECE
DEBUG=True                   # False en producción

# — BASE DE DATOS —
DATABASE_URL=postgresql+asyncpg://media_dev:PASSWORD@localhost:5432/media_db
# En Azure:
postgresql+asyncpg://USUARIO@SERVIDOR:PASSWORD@SERVIDOR.postgres.database.azure.com:5432/DBNAME?ssl=require

# — AUTENTICACIÓN JWT —
JWT_SECRET_KEY=GENERAR_CON_openssl_rand_hex_64
JWT_ALGORITHM=HS256
JWT_ACCESS_TOKEN_EXPIRE_MINUTES=15
JWT_REFRESH_TOKEN_EXPIRE_DAYS=7

# — 2FA / TOTP —
TOTP_ISSUER=MedIA-ECE
TOTP_ENCRYPTION_KEY=GENERAR_CON_Fernet.generate_key().decode()
# La clave Fernet cifra el totp_secret almacenado en usuarios_sistema

# — SEGURIDAD —
ALLOWED_ORIGINS=http://localhost:5173,https://media.azurestaticapps.net
MAX_LOGIN_ATTEMPTS=5
ACCOUNT_LOCKOUT_MINUTES=30

# — AZURE BLOB STORAGE —
AZURE_STORAGE_ACCOUNT_NAME=mediaece
AZURE_STORAGE_ACCOUNT_KEY=CLAVE_DE_AZURE_PORTAL
AZURE_BLOB_CONTAINER_LAB=lab-results           # PDFs de laboratorio externo
AZURE_BLOB_CONTAINER_TUTORES=tutores-docs      # Documentos de representantes legales
AZURE_BLOB_SAS_EXPIRY_MINUTES=15               # Tokens temporales de acceso

# — GENERACIÓN DE PDFs —
PDF_LOGO_PATH=./assets/logo_distrito_salud.png
PDF_FOOTER_TEXT=Distrito de Salud I - Tuxtla Gutierrez, Chiapas
PDF_HASH_ALGORITHM=SHA256                      # Req 1 Cómputo Forense

# — INFRAESTRUCTURA AZURE (Producción) —
AZURE_APP_SERVICE_URL=https://media-api.azurewebsites.net
AZURE_STATIC_WEB_APP_URL=https://media.azurestaticapps.net
APPLICATIONINSIGHTS_CONNECTION_STRING=DESDE_AZURE_PORTAL

# — POSTGRES LOCAL (Docker Compose) —
POSTGRES_DB=media_db
POSTGRES_USER=media_dev
POSTGRES_PASSWORD=SOLO_PARA_DESARROLLO_LOCAL
```

Frontend — .env.example

```
# =====  
# MedIA — Frontend Environment Variables (Vite)  
# Solo variables VITE_* son accesibles en el navegador.  
# NUNCA colocar secrets aquí.  
# =====  
  
VITE_API_URL=http://localhost:8000          # Backend en desarrollo  
# En producción: https://media-api.azurewebsites.net  
  
VITE_APP_NAME=MedIA  
VITE_APP_VERSION=1.0.0  
VITE_APP_ENV=development  
  
# Nombre del establecimiento por defecto (para header de notas)  
VITE_DEFAULT_CLUES=CSSSA023999           # CLUES de la unidad (seed)
```


5. Scripts de Arranque — Comandos Exactos

5.1 Primera vez (setup completo)

```
# 1. Clonar el repositorio
git clone https://github.com/unach-ece/media-ece.git
cd media-ece

# 2. Crear archivos de entorno
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env
# Editar los archivos .env con valores reales

# 3. Levantar PostgreSQL con Docker
docker-compose up -d postgres
# Esperar healthcheck: docker-compose ps (State: healthy)

# 4. Instalar dependencias del backend
cd backend
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt

# 5. Ejecutar seed inicial (catálogos + admin)
python -m app.database create_tables
# Los triggers y seeds se aplican automáticamente desde Docker init scripts

# 6. Instalar dependencias del frontend
cd ../frontend
npm install
```

5.2 Desarrollo diario

```
# Terminal 1 — PostgreSQL (si no está corriendo)
docker-compose up -d postgres

# Terminal 2 — Backend FastAPI
cd backend && source venv/bin/activate
uvicorn app.main:app --reload --port 8000
# Swagger disponible en: http://localhost:8000/docs

# Terminal 3 — Frontend Vite
cd frontend
npm run dev
# App disponible en: http://localhost:5173
```

5.3 Verificación del entorno

```
# Verificar PostgreSQL está listo
docker-compose ps          # State: healthy
docker exec media_db_dev psql -U media_dev -d media_db -c '\dt'
# Debe listar las 40 tablas

# Verificar backend
curl http://localhost:8000/health
# Respuesta esperada: {"status": "ok", "db": "connected"}

# Verificar usuario admin
curl -X POST http://localhost:8000/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username": "admin@media.local", "password": "MedIA2026!"}'
# Respuesta: {"access_token": "...", "requires_2fa": true}
```

Credenciales del seed: admin@media.local / MedIA2026! — CAMBIAR INMEDIATAMENTE después del primer login. El sistema forzará cambio de contraseña en el primer acceso.

6. Arquitectura de Distribución (Cómputo Distribuido)

MedIA implementa una arquitectura de base de datos distribuida funcional mediante los servicios gestionados de Azure, sin implementar protocolos de replicación propios. Este enfoque sigue el principio de no duplicar capacidad ya provista por la plataforma.

6.1 Topología de Distribución de Datos

Grupo de Entidades	Clasificación	Estrategia y Justificación
notas_medicas, pacientes, auditoria_accesos	CENTRALIZA DA	Datos maestros clínicos con consistencia fuerte requerida. Escritura siempre al servidor primario Azure (East US). Un paciente no puede tener dos versiones simultáneas.
cat_cie10, cat_medicamentos, cat_lenguas_indigenas	REPLICADA (lectura)	Catálogos estáticos de consulta frecuente. Cacheados en nodos CDN regionales de Azure Front Door. Invalidan manualmente cuando DGIS publica actualización del CIE-10.
cat_estados, cat_municipios, cat_localidades	REPLICADA (lectura)	Catálogos geográficos INEGI. Consulta frecuente en formularios de registro de pacientes. Sin cambios operacionales. Servidos desde CDN.
encuentros_clinicos, signos_vitales	CENTRALIZA DA + acceso distribuido	Escritura siempre al servidor central. Lectura distribuida vía Azure Front Door que enruta al nodo con menor latencia para el establecimiento solicitante.
jurisdicciones_sanitarias, establecimientos	REPLICADA (lectura)	Datos administrativos de referencia necesarios en todos los nodos para validar accesos y rutas de referencia médica.
auditoria_accesos, historial_cambios	CENTRALIZA DA Estricta	Jamás se replican ni distribuyen. Datos de auditoría médico-legal deben residir en un único nodo con control total. Trigger tr_audit_no_changes garantiza inmutabilidad.

Tabla 6.1 — Estrategia de distribución de datos por grupo de entidades

6.2 Diagrama de Arquitectura (Descripción)

La arquitectura de MedIA se compone de tres capas con comunicación definida:

Capa	Componentes y flujo de comunicación
CLIENTE	Navegador web del usuario en el establecimiento médico. Ejecuta el bundle Vite (React SPA). Comunicación HTTPS con Azure Front Door via JSON REST. JWT en memoria (no en disco). Sin acceso directo a PostgreSQL.
BACKEND API	Azure App Service (Python 3.11 + FastAPI). Recibe requests autenticados con JWT. Aplica middleware: auth → sanitize → audit → router → service. Genera PDFs con WeasyPrint. Interactúa con Azure Blob Storage para archivos externos. Expone /docs (Swagger) solo en desarrollo.
DATOS	Azure Database for PostgreSQL (servidor primario + réplica de lectura). Triggers PL/pgSQL activos. Azure Blob Storage para PDFs de laboratorio y documentos de tutores. Azure CDN para catálogos de solo lectura (invalidación manual).

Tabla 6.2 — Capas de la arquitectura del sistema

6.3 Sincronización y Descomposición de Consultas

Sincronización de catálogos

- Las escrituras clínicas se dirigen siempre al servidor primario (East US).
- Los catálogos replicados en CDN se invalidan mediante llamada al endpoint POST /admin/cache/invalidate cuando el coordinador actualiza un catálogo.
- El tiempo máximo de staleness de catálogos en CDN es de 24 horas, configurado en las reglas de caché de Azure Front Door.

Descomposición de la consulta 'Historial completo de paciente'

Pas o	Consulta	Fuente de datos
1	Datos demográficos básicos	Cache CDN si disponible (v_paciente_basico). Nunca incluye datos clínicos.
2	Alergias y antecedentes	Servidor primario PostgreSQL. Regla 1: accesibles para cualquier médico con encuentro activo.
3	Notas SOAP por encuentro	Servidor primario con validación de Regla 2 (referencia activa) y Regla 3 (especialidad del establecimiento). NUNCA se cachean.

Tabla 6.3 — Descomposición de consulta de historial completo

7. Estrategia de Ramas Git

La estrategia de ramas sigue el modelo GitFlow simplificado adaptado a un equipo de 5 personas en 9 semanas.

Rama	Regla y propósito
main	Producción. Protegida. Solo P1 puede mergear. Cada merge a main genera un tag de versión (v1.0, v1.1). Deployado automáticamente a Azure.
develop	Integración continua. Todas las features deben pasar lint (ruff + eslint) antes de crear PR hacia develop. P1 aprueba todos los PRs.
feature/p{N}-{modulo}	Rama por feature. Ej: feature/p2-referencias, feature/p3-soap. Vida máxima: 1 semana. Se cierra con PR a develop al entregar el módulo.
fix/p{N}-{descripcion}	Corrección de bug encontrado en develop. PR a develop con descripción del bug, causa raíz y solución.
hotfix/descripcion	Fix urgente en producción. Solo P1. Merge a main Y a develop. Genera tag de versión patch (v1.0.1).

Tabla 7.1 — Estrategia completa de ramas Git

Regla de protección: La rama main tiene branch protection activada en GitHub: requiere PR, mínimo 1 aprobación (P1), no se permite force push ni eliminación.